

Assignment Number:11.3

J.Telesh Hall:203a51812 B-26

Q.No.	Question	Expected Time to complete
1	Lab 11: Data Structures with AI Implementing Fundamental Data Structures using AI Assistance Lab Objectives: By the end of this lab, students will be able to: - Design and implement fundamental data structures in Python using AI assistance. - Effectively prompt AI tools (e.g., GitHub Copilot) for code generation, optimization, and documentation. - Understand and compare core data structures: Arrays, Linked Lists, Stacks, Queues, Priority Queues, Trees, and Graphs. - Improve code readability, efficiency, and maintainability using AI-generated suggestions. Learning Outcomes After completing this lab, students will be able to: - Apply appropriate data structures to solve real-world problems. - Analyze time and space complexity of different data structure operations. - Use AI tools responsibly to assist (not replace) logical thinking and problem-solving. - Validate, test, and refine AI-generated code.	Week6 - Wednesday
	Task 1: Smart Contact Manager (Arrays & Linked Lists) Scenario SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts. Tasks - Implement the contact manager using arrays (lists). - Implement the same functionality using a linked list for dynamic memory allocation. - Implement the following operations in both approaches: - Add a contact - Search for a contact - Delete a contact - Use GitHub Copilot to assist in generating search and delete methods. - Compare array vs. linked list approaches with respect to: - Insertion efficiency	

- Deletion efficiency

Expected Outcome

- Two working implementations (array-based and linked-list-based).
- A brief comparison explaining performance differences.

The screenshot displays a VS Code editor with two Python files: `array_contact_manager.py` and `linked_list_contact_manager.py`. The `array_contact_manager.py` file contains the following code:

```
1 class ArrayContactManager:
2     def __init__(self):
3         self.contacts = []
4     def add_contact(self, name, phone):
5         self.contacts.append({'name': name, 'phone': phone})
6     def search_contact(self, name):
7         for contact in self.contacts:
8             if contact['name'] == name:
9                 return contact['phone']
10        return None
11    def delete_contact(self, name):
12        for i, contact in enumerate(self.contacts):
13            if contact['name'] == name:
14                del self.contacts[i]
15            return True
16        return False
17
18 class Node:
19     def __init__(self, name, phone):
20         self.name = name
21         self.phone = phone
22         self.next = None
```

The terminal output shows the following commands and results:

```
PS C:\Users\shash\Downloads> conda activate shashidhar
PS C:\Users\shash\Downloads> python array_contact_manager.py
Array search Alice: 123
Array delete Bob: True
Array search Bob after delete: None
Linked search Charlie: 789
Linked delete Dave: True
Linked search Dave after delete: None
```

The chat window on the right contains the following text:

CONTACT MANAGER IMPLEMENTATION WITH ARRAYS AND LINKED L...

- Add a contact
- Search for a contact
- Delete a contact
- 4. Use GitHub Copilot to assist in generating search and delete methods.
- 5. Compare array vs. linked list approaches with respect to:
 - Insertion efficiency
 - Deletion efficiency
- Expected Outcome
 - Two working implementations (array-based and linked-list-based).
 - A brief comparison explaining performance differences.
- give code in python without comments

I've created three files in your current directory (C:\shash) with the implementations:

- `array_contact_manager.py`: Array-based contact manager
- `linked_list_contact_manager.py`: Linked list-based contact manager
- `comparison.txt`: Brief performance comparison

The code is provided without comments as requested. You can run these files directly in Python to test the functionality. For example, you could create a simple test script to instantiate the classes and call the methods.

```

Welcome AAC 11.4.py X
C: > Users > shash > Downloads > AAC 11.4.py > ArrayContactManager
18 class Node:
19     def __init__(self, name, phone):
20
21         self.phone = phone
22         self.next = None
23
24     class LinkedListContactManager:
25     def __init__(self):
26         self.head = None
27     def add_contact(self, name, phone):
28         new_node = Node(name, phone)
29         if not self.head:
30             self.head = new_node
31         else:
32             current = self.head
33             while current.next:
34                 current = current.next
35             current.next = new_node
36     def search_contact(self, name):
37         current = self.head
38         while current:
39             if current.name == name:
40                 return current.phone
41             current = current.next

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\shash\Downloads> conda activate Shashidhar
PS C:\Users\shash\Downloads> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\Downloads\AAC 11.4.py'
Array search Alice: 123
Array delete Bob: True
Array search Bob after delete: None
Linked search Charlie: 789
Linked delete Dave: True
Linked search Dave after delete: None
PS C:\Users\shash\Downloads>

```

```

Welcome AAC 11.4.py X
C: > Users > shash > Downloads > AAC 11.4.py > ArrayContactManager
24 class LinkedListContactManager:
36     def search_contact(self, name):
37
38         return current.phone
39         current = current.next
40     return None
41
42     def delete_contact(self, name):
43         if not self.head:
44             return False
45         if self.head.name == name:
46             self.head = self.head.next
47             return True
48         current = self.head
49         while current.next:
50             if current.next.name == name:
51                 current.next = current.next.next
52                 return True
53             current = current.next
54         return False
55
56     array_cm = ArrayContactManager()
57     array_cm.add_contact("Alice", "123")
58     array_cm.add_contact("Bob", "456")
59     print("Array search Alice:", array_cm.search_contact("Alice"))
60

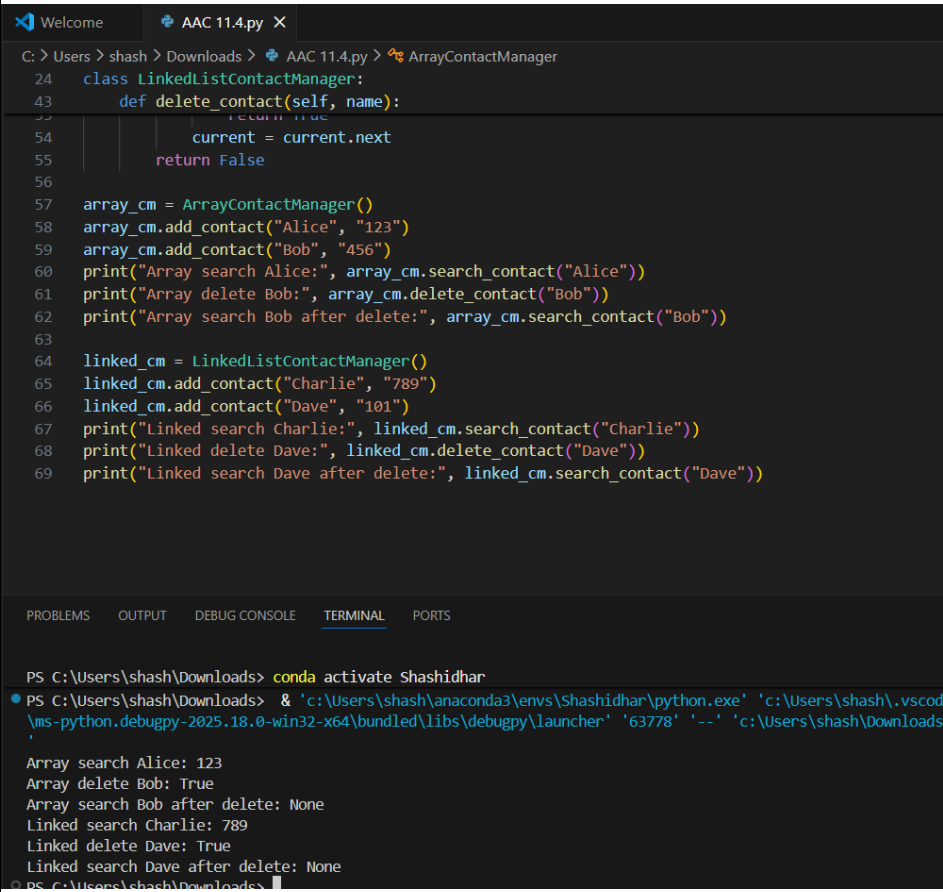
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\shash\Downloads> conda activate Shashidhar
PS C:\Users\shash\Downloads> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\Downloads\AAC 11.4.py'
Array search Alice: 123
Array delete Bob: True
Array search Bob after delete: None
Linked search Charlie: 789
Linked delete Dave: True
Linked search Dave after delete: None
PS C:\Users\shash\Downloads>

```

	 <pre> Welcome AAC 11.4.py X C: > Users > shash > Downloads > AAC 11.4.py > ArrayContactManager 24 class LinkedListContactManager: 43 def delete_contact(self, name): 54 current = current.next 55 return False 56 57 array_cm = ArrayContactManager() 58 array_cm.add_contact("Alice", "123") 59 array_cm.add_contact("Bob", "456") 60 print("Array search Alice:", array_cm.search_contact("Alice")) 61 print("Array delete Bob:", array_cm.delete_contact("Bob")) 62 print("Array search Bob after delete:", array_cm.search_contact("Bob")) 63 64 linked_cm = LinkedListContactManager() 65 linked_cm.add_contact("Charlie", "789") 66 linked_cm.add_contact("Dave", "101") 67 print("Linked search Charlie:", linked_cm.search_contact("Charlie")) 68 print("Linked delete Dave:", linked_cm.delete_contact("Dave")) 69 print("Linked search Dave after delete:", linked_cm.search_contact("Dave")) PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PS C:\Users\shash\Downloads> conda activate Shashidhar PS C:\Users\shash\Downloads> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63778' '--' 'c:\Users\shash\Downloads\ Array search Alice: 123 Array delete Bob: True Array search Bob after delete: None Linked search Charlie: 789 Linked delete Dave: True Linked search Dave after delete: None PS C:\Users\shash\Downloads> </pre>	
	Task 2: Library Book Search System (Queues & Priority Queues) Scenario The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests. Tasks 1. Implement a Queue (FIFO) to manage book requests. 2. Extend the system to a Priority Queue, prioritizing faculty requests. 3. Use GitHub Copilot to assist in generating: - o enqueue() method - o dequeue() method 4. Test the system with a mix of student and faculty requests. Expected Outcome • Working queue and priority queue implementations. • Correct prioritization of faculty requests.	

1234567891011121314151617181920212223242526272829303132333435363738394041424344

from collections import deque
import heapq
class BookRequest:
 def __init__(self, requester_name, book_title, role):
 self.requester_name = requester_name
 self.book_title = book_title
 self.role = role
 def __repr__(self):
 return f"({self.role.upper()}) | ({self.requester_name}) -> '{self.book_title}'"
class LibraryQueue:
 def __init__(self):
 self._queue = deque()
 def enqueue(self, request):
 self._queue.append(request)
 print(f"([QUEUE]) Added: {request}")
 def dequeue(self):
 if not self._queue:
 print("([QUEUE]) No requests in queue.")
 return None
 request = self._queue.popleft()
 print(f"([QUEUE]) Processing: {request}")
 return request
 def is_empty(self):
 return len(self._queue) == 0

PS C:\Users\shash\Downloads> cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda\envs\Shashidhar\python.exe' -i
'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '59531' '-...'
c:\Users\shash\Downloads\AAC_11.4.py'

--- Dequeuing all requests ---
[PRIORITY QUEUE] Processing: FACULTY | Dr. Smith -> 'Algorithms'
[PRIORITY QUEUE] Processing: FACULTY | Dr. Lee -> 'Database Systems'
[PRIORITY QUEUE] Processing: STUDENT | Alice -> 'Data Structures'
[PRIORITY QUEUE] Processing: STUDENT | Bob -> 'Operating Systems'
[PRIORITY QUEUE] Processing: STUDENT | Carol -> 'Computer Networks'
PS C:\Users\shash\Downloads>

CHAT

LIBRARY BOOK REQUEST SYSTEM IMPLEMENTATION

Library Book Search System (Queues & Priority Queues)
Scenario
The SRU library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.
Tasks
1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:
o enqueue() method
o dequeue() method
4. Test the system with a mix of student and faculty requests.
Expected Outcome
• Working queue and priority queue implementations.
• Correct prioritization of faculty requests.
give code in python without comments just paste in the file

✓ Optimizing tool selection...
✓ Reviewed and created AAC_11.4.py
✓ Created + AAC_11.4.py
✓ Configured a Python environment
✓ Ran pylance-lsp-mcp-server - pylance mcp server (MCP Server)
✓ Reviewed code and provided a brief summary

✚ AAC_11.4.py
Describe what to build next
Claude Sonnet 4.5

23242526272829303132333435363738394041424344

def is_empty(self):
 return len(self._queue) == 0
 def display(self):
 if not self._queue:
 print("([QUEUE]) Queue is empty.")
 else:
 print("([QUEUE]) Current requests:")
 for req in self._queue:
 print(f"({req})")
class PriorityLibraryQueue:
 PRIORITY_MAP = {"faculty": 0, "student": 1}
 def __init__(self):
 self._heap = []
 self._counter = 0
 def enqueue(self, request):
 priority = self.PRIORITY_MAP.get(request.role.lower(), 2)
 heapq.heappush(self._heap, (priority, self._counter, request))
 self._counter += 1
 print(f"([PRIORITY QUEUE]) Added: {request}")
 def dequeue(self):
 if not self._heap:

PS C:\Users\shash\Downloads> cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda\envs\Shashidhar\python.exe' -i
'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '59531' '-...'
c:\Users\shash\Downloads\AAC_11.4.py'

--- Dequeuing all requests ---
[PRIORITY QUEUE] Processing: FACULTY | Dr. Smith -> 'Algorithms'
[PRIORITY QUEUE] Processing: FACULTY | Dr. Lee -> 'Database Systems'
[PRIORITY QUEUE] Processing: STUDENT | Alice -> 'Data Structures'
[PRIORITY QUEUE] Processing: STUDENT | Bob -> 'Operating Systems'
[PRIORITY QUEUE] Processing: STUDENT | Carol -> 'Computer Networks'
PS C:\Users\shash\Downloads>

CHAT

LIBRARY BOOK REQUEST SYSTEM IMPLEMENTATION

✓ Optimizing tool selection...
✓ Reviewed and created AAC_11.4.py
✓ Created + AAC_11.4.py
✓ Configured a Python environment
✓ Ran pylance-lsp-mcp-server - pylance mcp server (MCP Server)
✓ Reviewed code and provided a brief summary

Code written to AAC_11.4.py(c:\Users\shash\Downloads\AAC_11.4.py) and verified working. Here's what the output shows:
Regular Queue (FIFO): Processes in insertion order — Alice, Dr. Smith, Bob, Dr. Lee, Carol.
Priority Queue: Automatically reorders so faculty always come first — Dr. Smith, Dr. Lee (faculty), then Alice, Bob, Carol (students). Within the same role, arrival order (FIFO) is preserved.
Key classes:
• BookRequest — holds requester name, book title, and role
• LibraryQueue — FIFO using collections.deque with enqueue()/dequeue()
• PriorityLibraryQueue — min-heap via heapq assigning priority @ to faculty and @ to students

✚ AAC_11.4.py
Describe what to build next
Claude Sonnet 4.5

```

Welcome AAC 11.4.py
C: > Users > shash > Downloads > AAC 11.4.py > ...
32 class PriorityLibraryQueue:
43     def dequeue(self):
45         print("[PRIORITY QUEUE] No requests in queue.")
46         return None
47         _, _ , request = heapq.heappop(self._heap)
48         print(f"[PRIORITY QUEUE] Processing: {request}")
49         return request
50     def is_empty(self):
51         return len(self._heap) == 0
52     def display(self):
53         if not self._heap:
54             print("[PRIORITY QUEUE] Queue is empty.")
55         else:
56             sorted_requests = sorted(self._heap, key=lambda x: (x[0], x[1]))
57             print("[PRIORITY QUEUE] Current requests (by priority):")
58             for _, _, req in sorted_requests:
59                 print(f" {req}")
60 print("=" * 55)
61 print("          REGULAR QUEUE (FIFO) TEST")
62 print("=" * 55)
63 library_queue = LibraryQueue()
64 library_queue.enqueue(BookRequest("Alice", "Data Structures", "student"))
65 library_queue.enqueue(BookRequest("Dr. Smith", "Algorithms", "faculty"))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\Downloads> c::; cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda3\envs\S
'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\laur
:\Users\shash\Downloads\AAC 11.4.py'

--- Dequeuing all requests ---
[PRIORITY QUEUE] Processing: FACULTY | Dr. Smith -> 'Algorithms'
[PRIORITY QUEUE] Processing: FACULTY | Dr. Lee -> 'Database Systems'
[PRIORITY QUEUE] Processing: STUDENT | Alice -> 'Data Structures'
[PRIORITY QUEUE] Processing: STUDENT | Bob -> 'Operating Systems'
[PRIORITY QUEUE] Processing: STUDENT | Carol -> 'Computer Networks'
PS C:\Users\shash\Downloads>

```

```

C:\Users\shash\Downloads> AAC 11.4.py > ...
65 library_queue.enqueue(BookRequest("Dr. Smith", "Algorithms", "faculty"))
66 library_queue.enqueue(BookRequest("Bob", "Operating Systems", "student"))
67 library_queue.enqueue(BookRequest("Dr. Lee", "Database Systems", "faculty"))
68 library_queue.enqueue(BookRequest("Carol", "Computer Networks", "student"))
69 print()
70 library_queue.display()
71 print()
72 print("--- Dequeuing all requests ---")
73 while not library_queue.is_empty():
74     library_queue.dequeue()
75 print()
76 print("=" * 55)
77 print("        PRIORITY QUEUE TEST (Faculty First)")
78 print("=" * 55)
79 priority_queue = PriorityLibraryQueue()
80 priority_queue.enqueue(BookRequest("Alice", "Data Structures", "student"))
81 priority_queue.enqueue(BookRequest("Dr. Smith", "Algorithms", "faculty"))
82 priority_queue.enqueue(BookRequest("Bob", "Operating Systems", "student"))
83 priority_queue.enqueue(BookRequest("Dr. Lee", "Database Systems", "faculty"))
84 priority_queue.enqueue(BookRequest("Carol", "Computer Networks", "student"))
85 print()
86 priority_queue.display()
87 print()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

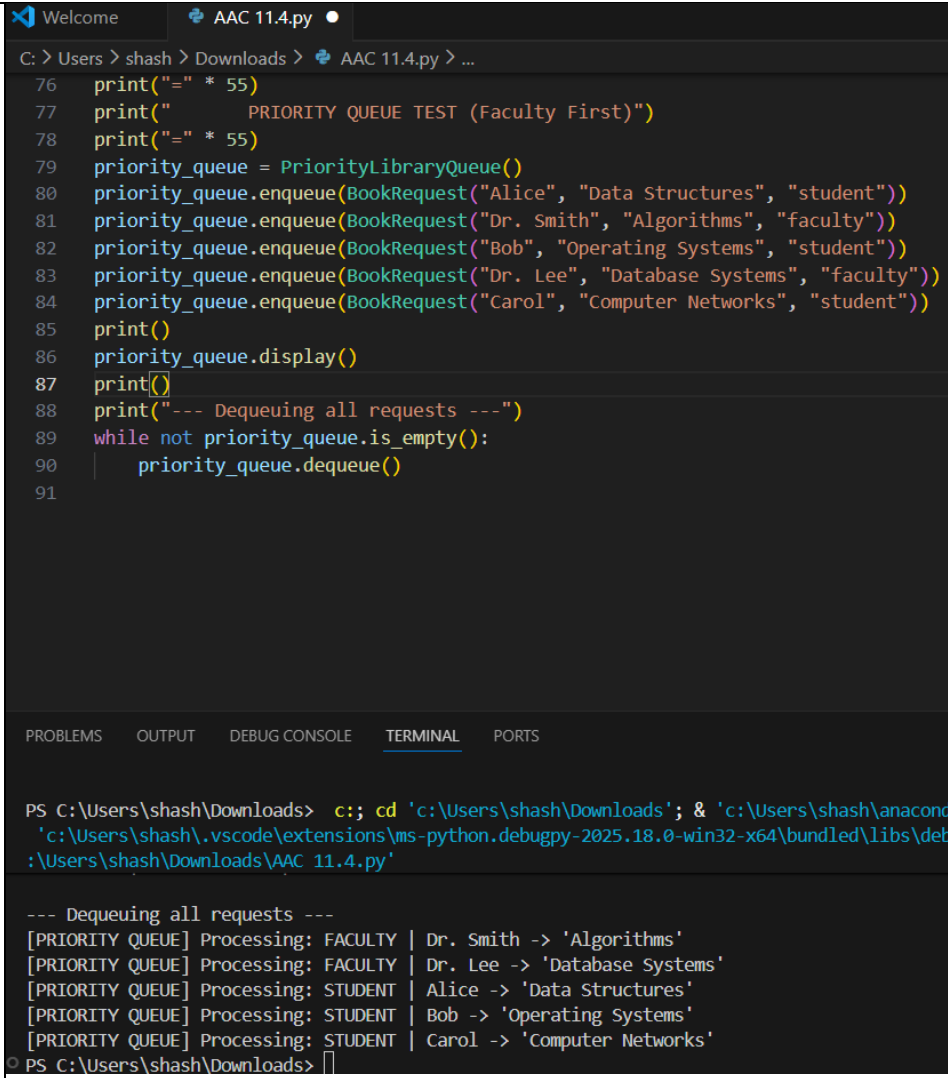
PS C:\Users\shash\Downloads> c:; cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda3\
'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy
:\Users\shash\Downloads\AAC 11.4.py'

```

```

--- Dequeuing all requests ---
[PRIORITY QUEUE] Processing: FACULTY | Dr. Smith -> 'Algorithms'
[PRIORITY QUEUE] Processing: FACULTY | Dr. Lee -> 'Database Systems'
[PRIORITY QUEUE] Processing: STUDENT | Alice -> 'Data Structures'
[PRIORITY QUEUE] Processing: STUDENT | Bob -> 'Operating Systems'
[PRIORITY QUEUE] Processing: STUDENT | Carol -> 'Computer Networks'

```

	 <pre> 76 print("=" * 55) 77 print(" PRIORITY QUEUE TEST (Faculty First)") 78 print("=" * 55) 79 priority_queue = PriorityLibraryQueue() 80 priority_queue.enqueue(BookRequest("Alice", "Data Structures", "student")) 81 priority_queue.enqueue(BookRequest("Dr. Smith", "Algorithms", "faculty")) 82 priority_queue.enqueue(BookRequest("Bob", "Operating Systems", "student")) 83 priority_queue.enqueue(BookRequest("Dr. Lee", "Database Systems", "faculty")) 84 priority_queue.enqueue(BookRequest("Carol", "Computer Networks", "student")) 85 print() 86 priority_queue.display() 87 print() 88 print("--- Dequeuing all requests ---") 89 while not priority_queue.is_empty(): 90 priority_queue.dequeue() 91 </pre> <pre> PS C:\Users\shash\Downloads> c::; cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anacond 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\deb :\Users\shash\Downloads\AAC 11.4.py' --- Dequeuing all requests --- [PRIORITY QUEUE] Processing: FACULTY Dr. Smith -> 'Algorithms' [PRIORITY QUEUE] Processing: FACULTY Dr. Lee -> 'Database Systems' [PRIORITY QUEUE] Processing: STUDENT Alice -> 'Data Structures' [PRIORITY QUEUE] Processing: STUDENT Bob -> 'Operating Systems' [PRIORITY QUEUE] Processing: STUDENT Carol -> 'Computer Networks' PS C:\Users\shash\Downloads> </pre>	
	Task 3: Emergency Help Desk (Stack Implementation) Scenario SR University’s IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach. Tasks 1. Implement a Stack to manage support tickets. 2. Provide the following operations: - o push(ticket) - o pop() - o peek() 3. Simulate at least five tickets being raised and resolved. 4. Use GitHub Copilot to suggest additional stack operations such as: - o Checking whether the stack is empty - o Checking whether the stack is full (if applicable) Expected Outcome • Functional stack-based ticket management system. • Clear demonstration of LIFO behavior.	

WelcomeAAC 11.4py X

C:\Users\shash> shash > Downloads > AAC 11.4py > ...

```
1 class SupportTicket:
2     def __init__(self, ticket_id, requester, issue, priority="normal"):
3         self.ticket_id = ticket_id
4         self.requester = requester
5         self.issue = issue
6         self.priority = priority
7
8     def __repr__(self):
9         return f"[#{self.ticket_id}] {self.requester} | {self.issue} ({self.priority.upper()}]"
10
11
12 class HelpDeskStack:
13     MAX_SIZE = 10
14
15     def __init__(self):
16         self._stack = []
17
18     def push(self, ticket):
19         if self.is_full():
20             print(f"[STACK FULL] Cannot add: {ticket}")
21             return
22         self._stack.append(ticket)
23         print(f"[PUSHED] {ticket}")
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\Downloads> cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda\envs\shashidhar\python.exe' 'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51957' "... 'c:\Users\shash\Downloads\AAC 11.4.py'
SR UNIVERSITY IT HELP DESK - TICKET SYSTEM

--- Raising Support Tickets ---
[PUSHED] [#101] Alice (Student) | Cannot access Wi-Fi (NORMAL)
[PUSHED] [#102] Bob (Staff) | Projector not working in Lab3 (HIGH)
[PUSHED] [#103] Carol (Student) | Email login failure (NORMAL)
[PUSHED] [#104] Dr. Smith (Staff) | VPN disconnecting repeatedly (CRITICAL)

Python Debug Console

Classes:

- SupportTicket — holds ticket ID, requester, issue description, and priority
- HelpDeskStack — list-based stack with MAX_SIZE = 10

Created AAC 11.5.py

Created AAC 11.5.py

Tried to run command but was blocked

Run pylancechatcodeinterpreter - pylance mcp server (MCP Server)

Summarized results for the user

Code saved to [AAC 11.5.py](C:\Users\shash\Downloads\AAC 11.5.py) and verified working. Here's what it demonstrates:

WelcomeAAC 11.4py X

C:\Users\shash> shash > Downloads > AAC 11.4py > ...

```
12 class HelpDeskStack:
23     print(f"[PUSHED] {ticket}")
24
25     def pop(self):
26         if self.is_empty():
27             print(f"[STACK EMPTY] No tickets to resolve.")
28             return None
29         ticket = self._stack.pop()
30         print(f"[RESOLVED] {ticket}")
31         return ticket
32
33     def peek(self):
34         if self.is_empty():
35             print(f"[STACK EMPTY] No tickets in stack.")
36             return None
37         ticket = self._stack[-1]
38         print(f"[PEEK] Next to resolve: {ticket}")
39         return ticket
40
41     def is_empty(self):
42         return len(self._stack) == 0
43
44     def is_full(self):
45
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\Downloads> cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda\envs\shashidhar\python.exe' 'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51957' "... 'c:\Users\shash\Downloads\AAC 11.4.py'
[PUSHED] [#105] Eve (Student) | Printer offline in Library (NORMAL)

[STACK] Current tickets (top - most recent, total=5):
[#105] Eve (Student) | Printer offline in Library (NORMAL)
[#104] Dr. Smith (Staff) | VPN disconnecting repeatedly (CRITICAL)
[#103] Carol (Student) | Email login failure (NORMAL)
[#102] Bob (Staff) | Projector not working in Lab3 (HIGH)
[#101] Alice (Student) | Cannot access Wi-Fi (NORMAL)

Python Debug Console

Classes:

- SupportTicket — holds ticket ID, requester, issue description, and priority
- HelpDeskStack — list-based stack with MAX_SIZE = 10

Operations Implemented:

Method	Description
push(ticket)	Adds ticket to top of stack
pop()	Removes and resolves ticket from top (LIFO)
peek()	Views top ticket without removing it
is_empty()	Returns True if no tickets
is_full()	Returns True if at MAX_SIZE
size()	Returns current ticket count
display()	Shows all tickets top-to-bottom

Created AAC 11.4py

Describe what to build next

Claude Sonnet 4.6 v1

WelcomeAAC 11.4py X

C:\Users\shash> shash > Downloads > AAC 11.4py > ...

```
17 class HelpDeskStack:
40
41     def is_empty(self):
42         return len(self._stack) == 0
43
44     def is_full(self):
45         return len(self._stack) >= self.MAX_SIZE
46
47     def size(self):
48         return len(self._stack)
49
50     def display(self):
51         if self.is_empty():
52             print(f"[STACK] Stack is empty.")
53         else:
54             print(f"[STACK] Current tickets (top - most recent, total={self.size}):")
55             for ticket in reversed(self._stack):
56                 print(f"  {ticket}")
57
58     print(f"--- * 660")
59     print(f"SR UNIVERSITY IT HELP DESK - TICKET SYSTEM")
60     print(f"--- * 660")
61
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\Downloads> cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda\envs\shashidhar\python.exe' 'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51957' "... 'c:\Users\shash\Downloads\AAC 11.4.py'
[#101] Alice (Student) | Cannot access Wi-Fi (NORMAL)

[INFO] Stack empty? False
[INFO] Stack full? False
[INFO] Total tickets in stack: 5

--- Peeking at top ticket ---
[PEEK] Next to resolve: [#105] Eve (Student) | Printer offline in Library (NORMAL)

Python Debug Console

Classes:

- SupportTicket — holds ticket ID, requester, issue description, and priority
- HelpDeskStack — list-based stack with MAX_SIZE = 10

Operations Implemented:

Method	Description
push(ticket)	Adds ticket to top of stack
pop()	Removes and resolves ticket from top (LIFO)
peek()	Views top ticket without removing it
is_empty()	Returns True if no tickets
is_full()	Returns True if at MAX_SIZE
size()	Returns current ticket count
display()	Shows all tickets top-to-bottom

LIFO behavior confirmed: Tickets were pushed in order 101–105, and resolved in reverse order 105–101. The final pop() on an empty stack is handled gracefully with a message instead of crashing.

Created AAC 11.4py

Describe what to build next

Claude Sonnet 4.6 v1

```
Welcome AAC 11.4.py X
C: > Users > shash > Downloads > AAC 11.4.py > ...
58
59 print("=" * 60)
60 print("          SR UNIVERSITY IT HELP DESK - TICKET SYSTEM")
61 print("=" * 60)
62
63 help_desk = HelpDeskStack()
64
65 print("\n--- Raising Support Tickets ---")
66 help_desk.push(SupportTicket(101, "Alice (Student)", "Cannot access Wi-Fi", "normal"))
67 help_desk.push(SupportTicket(102, "Bob (Staff)", "Projector not working in Lab3", "high"))
68 help_desk.push(SupportTicket(103, "Carol (Student)", "Email login failure", "normal"))
69 help_desk.push(SupportTicket(104, "Dr. Smith (Staff)", "VPN disconnecting repeatedly", "critical"))
70 help_desk.push(SupportTicket(105, "Eve (Student)", "Printer offline in Library", "normal"))
71
72 print()
73 help_desk.display()
74
75 print(f"\n[INFO] Stack empty? {help_desk.is_empty()}")
76 print(f"[INFO] Stack full? {help_desk.is_full()}")
77 print(f"[INFO] Total tickets in stack: {help_desk.size()}")
78
79 print("\n--- Peeking at top ticket ---")
80 help_desk.peek()
81
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\Downloads> c::; cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe'
'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51957' '---'
:\Users\shash\Downloads\AAC 11.4.py'
--- Peeking at top ticket ---
[PEEK] Next to resolve: [#105] Eve (Student) | Printer offline in Library (NORMAL)

--- Resolving Tickets (LIFO Order) ---
[RESOLVED] [#105] Eve (Student) | Printer offline in Library (NORMAL)
[RESOLVED] [#104] Dr. Smith (Staff) | VPN disconnecting repeatedly (CRITICAL)
[RESOLVED] [#103] Carol (Student) | Email login failure (NORMAL)
[RESOLVED] [#102] Bob (Staff) | Projector not working in Lab3 (HIGH)
```

```
C: > Users > shash > Downloads > AAC 11.4.py > ...
72     print()
73     help_desk.display()
74
75     print(f"\n[INFO] Stack empty? {help_desk.is_empty()}")
76     print(f"[INFO] Stack full? {help_desk.is_full()}")
77     print(f"[INFO] Total tickets in stack: {help_desk.size()}")
78
79     print("\n--- Peeking at top ticket ---")
80     help_desk.peak()
81
82     print("\n--- Resolving Tickets (LIFO Order) ---")
83     while not help_desk.is_empty():
84         help_desk.pop()
85
86     print()
87     print(f"[INFO] Stack empty after resolving all? {help_desk.is_empty()}")
88     help_desk.pop()
89
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\Downloads> c++; cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\an
'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib
:\Users\shash\Downloads\AAC 11.4.py'
[RESOLVED] [#104] Dr. Smith (Staff) | VPN disconnecting repeatedly (CRITICAL)
[RESOLVED] [#103] Carol (Student) | Email login failure (NORMAL)
[RESOLVED] [#102] Bob (Staff) | Projector not working in Lab3 (HIGH)
[RESOLVED] [#101] Alice (Student) | Cannot access Wi-Fi (NORMAL)

[INFO] Stack empty after resolving all? True
[STACK EMPTY] No tickets to resolve.
PS C:\Users\shash\Downloads>
```

Task 4: Hash Table

Objective

To implement a **Hash Table** and understand collision handling.

Task Description

Use AI to generate a hash table with:

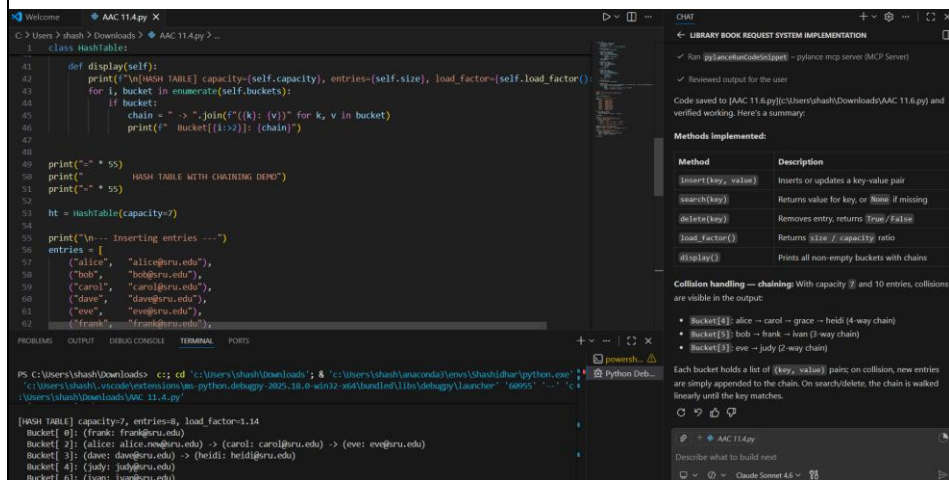
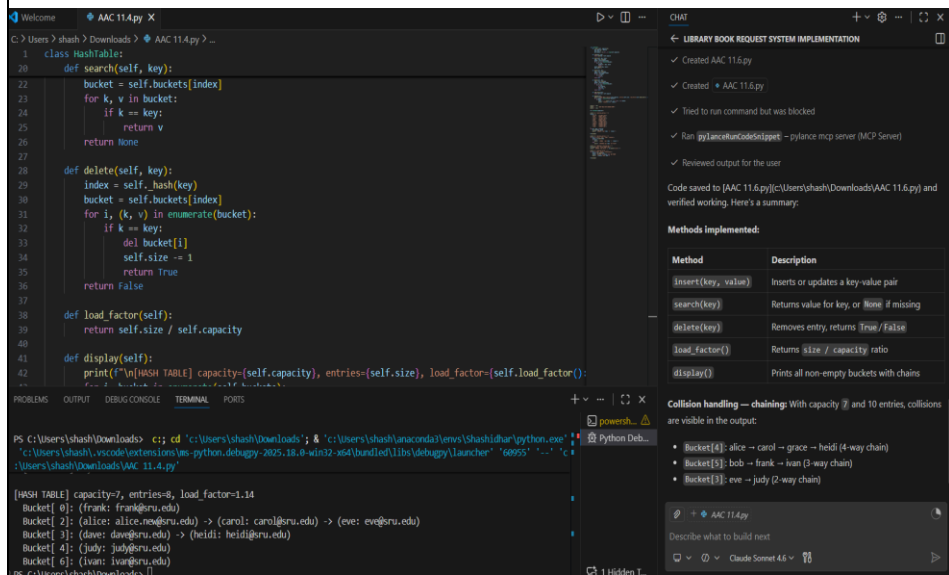
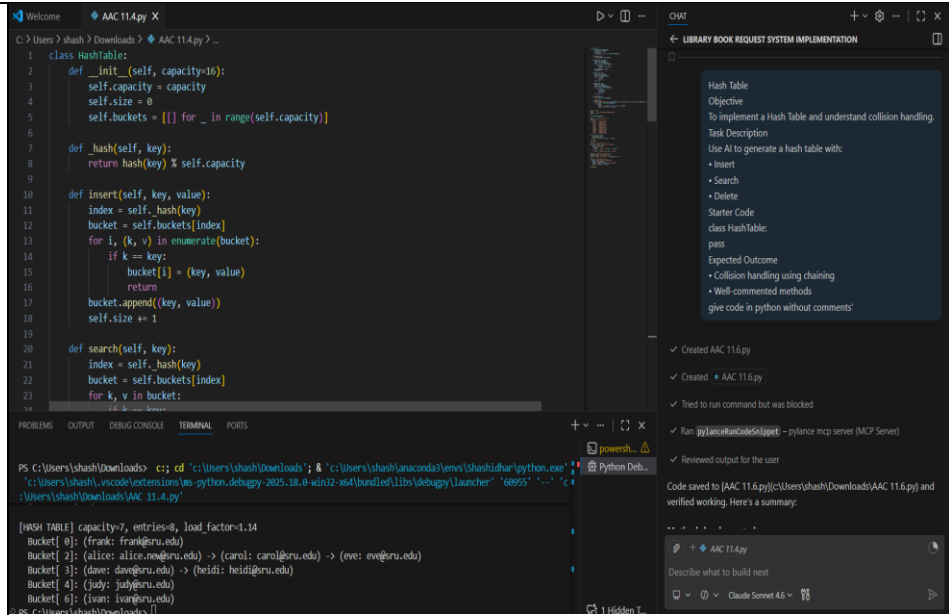
- Insert
- Search
- Delete

Starter Code

```
class HashTable:
    pass
```

Expected Outcome

- Collision handling using chaining
- Well-commented methods



C: > Users > shash > Downloads > AAC 11.4.py > ...

```
58     ("bob", "bob@sru.edu"),
59     ("carol", "carol@sru.edu"),
60     ("dave", "dave@sru.edu"),
61     ("eve", "eve@sru.edu"),
62     ("frank", "frank@sru.edu"),
63     ("grace", "grace@sru.edu"),
64     ("heidi", "heidi@sru.edu"),
65     ("ivan", "ivan@sru.edu"),
66     ("judy", "judy@sru.edu"),
67 ]
68 for key, value in entries:
69     ht.insert(key, value)
70     print(f" [INSERT] key='{key}' -> '{value}''")
71
72 ht.display()
73
74 print("\n--- Searching entries ---")
75 for key in ["alice", "eve", "judy", "unknown"]:
76     result = ht.search(key)
77     if result:
78         print(f" [FOUND] key='{key}' -> '{result}''")
79     else:
80         print(f" [MISSING] key='{key}' not found")
81
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\Downloads> c;; cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda3\Scripts\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\debugpy_launcher.py' 'c:\Users\shash\Downloads\AAC 11.4.py'

```
[HASH TABLE] capacity=7, entries=8, load_factor=1.14
Bucket[ 0]: (frank: frank@sru.edu)
Bucket[ 2]: (alice: alice.new@sru.edu) -> (carol: carol@sru.edu) -> (eve: eve@sru.edu)
Bucket[ 3]: (dave: dave@sru.edu) -> (heidi: heidi@sru.edu)
Bucket[ 4]: (judy: judy@sru.edu)
Bucket[ 6]: (ivan: ivan@sru.edu)
```

```
C:\Users\shash> Downloads > AAC 11.4.py > ...
75 for key in ["alice", "eve", "judy", "unknown"]:
76     result = ht.search(key)
77     if result:
78         print(f" [FOUND] key='{key}' -> '{result}'")
79     else:
80         print(f" [MISSING] key='{key}' not found")
81
82 print("\n--- Updating an existing key ---")
83 ht.insert("alice", "alice.new@sru.edu")
84 print(f" [UPDATE] key='alice' -> '{ht.search('alice')}'")
85
86 print("\n--- Deleting entries ---")
87 for key in ["bob", "grace", "nonexistent"]:
88     success = ht.delete(key)
89     status = "DELETED" if success else "NOT FOUND"
90     print(f" [{status}] key='{key}'")
91
92 ht.display()
93
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\Downloads> c:: cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anacond
'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\deb
:\Users\shash\Downloads\AAC 11.4.py'
```

```
[HASH TABLE] capacity=7, entries=8, load_factor=1.14
Bucket[ 0]: (frank: frank@sru.edu)
Bucket[ 2]: (alice: alice.new@sru.edu) -> (carol: carol@sru.edu) -> (eve: eve@sru.edu)
Bucket[ 3]: (dave: dave@sru.edu) -> (heidi: heidi@sru.edu)
Bucket[ 4]: (judy: judy@sru.edu)
Bucket[ 6]: (ivan: ivan@sru.edu)
PS C:\Users\shash\Downloads>
```

Task 5: Real-Time Application Challenge

Scenario

Design a **Campus Resource Management System** with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in **2–3 sentences**.
3. Implement **one selected feature** using AI-assisted code generation.

Expected Outcome

- Mapping table: Feature → Data Structure → Justification
- One fully working Python implementation

```
Welcome AAC 11.4.py AAC 11.7.py X
C:\Users\shash>Downloads> AAC 11.7.py > ...
1 from collections import deque
2 import heapq
3 from datetime import datetime
4
5 print("MAPPING: Feature -> Data Structure -> Justification")
6 print("-" * 60)
7 print("Attendance -> Dictionary -> O(1) Lookup by student ID")
8 print("Event Reg -> Set/Dictionary -> Deduplication + fast access")
9 print("Book Borrow -> Queue (FIFO) -> Serve requests in order")
10 print("Bus Schedule -> Priority Queue -> Earliest departure first")
11 print("Cafeteria -> Queue (FIFO) -> Orders prepared in sequence")
12
13 print("\n-- 1. Attendance (Dictionary) --")
14 attendance = {}
15 for sid, name, status in [("S001", "Alice", "present"), ("S002", "Bob", "absent"), ("S003", "Carol", "present"), ("S004", "Dave", "absent"), ("S005", "Eve", "present")]:
16     attendance.setdefault("2026-02-19", {})[sid] = (name, status)
17     print(f"({sid}) {name} -> {status}")
18 print("Report:", {sid: s for sid, (n, s) in attendance["2026-02-19"].items()})
19
20 print("\n-- 2. Event Registration (Set + Dict) --")
21 events = {"Tech Fest 2026": {"attendees": set(), "capacity": 5}}
22 for sid in ["S001", "S002", "S003", "S002", "S004", "S005", "S006"]:
23     e = events["Tech Fest 2026"]
24     if sid in e["attendees"]: print(f" DUPLICATE: {sid}")
25     elif len(e["attendees"]) >= e["capacity"]: print(f" FULL: {sid} rejected")
26     else: e["attendees"].add(sid); print(f" Registered: {sid}")
27 print("Attendees:", sorted(events["Tech Fest 2026"]["attendees"]))
28
29 print("\n-- 3. Library Borrow (Queue) --")
30 bq = deque([("S001", "Data Structures"), ("S003", "Operating Systems"), ("S005", "Computer Networks"), ("S002", "Data Structures")])
31 while bq:
32     sid, book = bq.popleft()
33     print(f" Issued '{book}' to {sid}")
34
35 print("\n-- 4. Bus Scheduling (Priority Queue) --")
36 buses, ctr = [], 0
37 for route, t in [("City", "07:30"), ("Airport", "09:00"), ("Mall", "08:15"), ("Station", "07:00"), ("Hospital", "10:30")]:
38     heapq.heappush(buses, (datetime.strptime(t, "%H:%M"), ctr, route)); ctr += 1
39 while buses:
40     dt, c, route = heapq.heappop(buses)
41     print(f" Next bus: {route} at {dt.strftime('%H:%M')} (ID: {c})")
42
43 print("\n-- 5. Cafeteria Orders (Queue) --")
44 orders = deque(["Burger for Alice", "Pasta for Bob", "Sandwich for Carol", "Rice for Dave", "Noodles for Eve"])
45 while orders:
46     order = orders.popleft()
47     print(f" Served: {order}")
48
49 PS C:\Users\shash\Downloads> c:\cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda3\envs\shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50953' '-.' 'c:\Users\shash\Downloads\AAC 11.7.py'
```

LIBRARY BOOK REQUEST SYSTEM IMPLEMENTATION

Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2-3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

- Mapping table: Feature -> Data Structure -> Justification
- One fully working Python implementation

give code in python without comments

Created AAC 11.7.py

Created AAC 11.7.py

Try to run command to verify

shrink the code

AAC 11.7.py

Describe what to build next

Claude Sonnet 4.6

```
Welcome AAC 11.4.py AAC 11.7.py X
C:\Users\shash>Downloads> AAC 11.7.py > ...
17 print(f"({sid}) {name} -> {status}")
18 print("Report:", {sid: s for sid, (n, s) in attendance["2026-02-19"].items()})
19
20 print("\n-- 2. Event Registration (Set + Dict) --")
21 events = {"Tech Fest 2026": {"attendees": set(), "capacity": 5}}
22 for sid in ["S001", "S002", "S003", "S002", "S004", "S005", "S006"]:
23     e = events["Tech Fest 2026"]
24     if sid in e["attendees"]: print(f" DUPLICATE: {sid}")
25     elif len(e["attendees"]) >= e["capacity"]: print(f" FULL: {sid} rejected")
26     else: e["attendees"].add(sid); print(f" Registered: {sid}")
27 print("Attendees:", sorted(events["Tech Fest 2026"]["attendees"]))
28
29 print("\n-- 3. Library Borrow (Queue) --")
30 bq = deque([("S001", "Data Structures"), ("S003", "Operating Systems"), ("S005", "Computer Networks"), ("S002", "Data Structures")])
31 while bq:
32     sid, book = bq.popleft()
33     print(f" Issued '{book}' to {sid}")
34
35 print("\n-- 4. Bus Scheduling (Priority Queue) --")
36 buses, ctr = [], 0
37 for route, t in [("City", "07:30"), ("Airport", "09:00"), ("Mall", "08:15"), ("Station", "07:00"), ("Hospital", "10:30")]:
38     heapq.heappush(buses, (datetime.strptime(t, "%H:%M"), ctr, route)); ctr += 1
39 while buses:
40     dt, c, route = heapq.heappop(buses)
41     print(f" Next bus: {route} at {dt.strftime('%H:%M')} (ID: {c})")
42
43 print("\n-- 5. Cafeteria Orders (Queue) --")
44 orders = deque(["Burger for Alice", "Pasta for Bob", "Sandwich for Carol", "Rice for Dave", "Noodles for Eve"])
45 while orders:
46     order = orders.popleft()
47     print(f" Served: {order}")
48
49 PS C:\Users\shash\Downloads> c:\cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda3\envs\shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50953' '-.' 'c:\Users\shash\Downloads\AAC 11.7.py'
```

```

Welcome  AAC 11.4.py  AAC 11.7.py X
C:\Users\shash> Downloads > AAC 11.7.py > ...
33     print(f" Issued '{book}' to {sid}")
34
35     print("\n-- 4. Bus Scheduling (Priority Queue) --")
36     buses, ctr = [], 0
37     for route, t in [("City", "07:30"), ("Airport", "09:00"), ("Mall", "08:15"), ("Station", "07:00"), ("Hospital", "10:30")
38         : heapq.heappush(buses, (datetime.strptime(t, "%H:%M"), ctr, route)); ctr += 1
39     while buses:
40         dt, _, r = heapq.heappop(buses)
41         print(f" Dispatch: {r} at {dt.strftime('%H:%M')}")
42
43     print("\n-- 5. Cafeteria Orders (Queue) --")
44     oq = deque(enumerate(["Alice:Burger", "Bob:Pasta", "Carol:Sandwich", "Dave:Rice", "Eve:Noodles"], 1))
45     while oq:
46         num, entry = oq.popleft()
47         name, item = entry.split(":")
48         print(f" Order #{num} ready: {item} for {name}")
49
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\shash\Downloads> c;; cd 'c:\Users\shash\Downloads'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe'
'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50953' '--' 'c
:\Users\shash\Downloads\AAC 11.7.py'
MAPPING: Feature -> Data Structure -> Justification
-- 5. Cafeteria Orders (Queue) --
Order #1 ready: Burger for Alice
Order #2 ready: Pasta for Bob
Order #3 ready: Sandwich for Carol
Order #4 ready: Rice for Dave
Order #5 ready: Noodles for Eve
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.